

Project : Refactoring and Testing of Java Calculator Application

Sw340

Name	ID
Dalia fahad	445009179
Khlood alshamrani	445009188
Maya Alshehri	445009190
Shahad Alyaseen	445009193

1. Introduction

Software Quality Assurance (SQA) plays a vital role in ensuring that software systems are developed according to high standards of correctness, maintainability, and reliability. This project was conducted as part of the SW340 course to apply SQA techniques to a simple but flawed Java application — a basic calculator program intentionally designed with logical and structural errors.

The main objective of this project is to detect defects, analyze bad coding practices, and apply software quality improvements such as refactoring, input validation, modularization, and automated testing. By improving a simple calculator, we demonstrate how SQA principles transform poor-quality code into a maintainable, reliable, and testable software product.

2. Tools and Environment

Tool / Technology	Purpose
Java (JDK 17 – Azul Zulu 25)	Programming Language
Apache Maven	Build and Dependency Management
IntelliJ IDEA	Development Environment
JUnit 5	Unit Testing Framework
Windows 10 / 11	Operating System

A Maven-based structure was adopted to standardize dependencies, testing, and compilation. This environment also ensured that all team members could build and test the system consistently across devices.

3. Methodology and Process

The project followed a structured SQA process involving the following phases:

1.Code Analysis:

The initial “Bad Calculator” code was examined to identify logical, syntactic, and structural flaws. The team documented all observed issues such as missing input validation, poor naming conventions, and lack of testing.

2.Refactoring:

Based on the analysis, the team reorganized the program into smaller, modular components following clean code principles. The logic was separated into a dedicated Calculator class, while user interaction was handled by App.java.

3.Testing (Verification and Validation):

After refactoring, unit tests were written using JUnit 5 to verify the correctness of each arithmetic function. These tests provided formal validation that each requirement (addition, subtraction, multiplication, division, and exception handling) works as expected.

4.Documentation and Reporting:

Screenshots, test results, and comparative analysis were compiled to provide traceability and evidence for each improvement step.

4. Problem Identification (Before Refactoring)

The original Bad Calculator contained numerous software quality problems, summarized below:

Issue	Description
Logic Errors	Used == for comparing strings, causing operations to fail.
No Input Validation	Division by zero was not handled, resulting in "Infinity."
Mixed Responsibilities	Logic and user input/output in a single main method.
Poor Readability	Minimal structure and unclear variable naming.
No Testing	No automated or manual validation mechanisms.

Figure 1: Original BadCalculator.java before refactoring.

```
1 package org.example;
2 import java.util.*;
3
4 public class BadCalculator {
5
6     public static void main(String args[]) {
7
8         Scanner s = new Scanner(System.in);
9
10        double num1, num2, result = 0;
11        String operator;
12
13        System.out.println("Enter first number: ");
14        num1 = s.nextInt();
15
16        System.out.println("Enter operator: ");
17        operator = s.next(); //
18
19        System.out.println("Enter second number: ");
20        num2 = s.nextInt(); //
21
22        if (operator == "+") {
23            result = num1 + num2;
24        } else if (operator == "-") {
25            result = num1 - num2;
26        } else if (operator == "*") {
27            result = num1 * num2;
28        } else if (operator == "/") {
29            result = num1 / num2;
30        }
31    }
32}
```

```
4 public class BadCalculator {
5
6     public static void main(String args[]) {
7
8         Scanner s = new Scanner(System.in);
9
10        double num1, num2, result = 0;
11        String operator;
12
13        System.out.println("Enter first number: ");
14        num1 = s.nextInt();
15
16        System.out.println("Enter operator: ");
17        operator = s.next(); //
18
19        System.out.println("Enter second number: ");
20        num2 = s.nextInt(); //
21
22        if (operator == "+") {
23            result = num1 + num2;
24        } else if (operator == "-") {
25            result = num1 - num2;
26        } else if (operator == "*") {
27            result = num1 * num2;
28        } else if (operator == "/") {
29            result = num1 / num2;
30        } else {
31            System.out.println("invalid operator");
32        }
33
34        System.out.println("Result: " + result);
35    }
36}
```

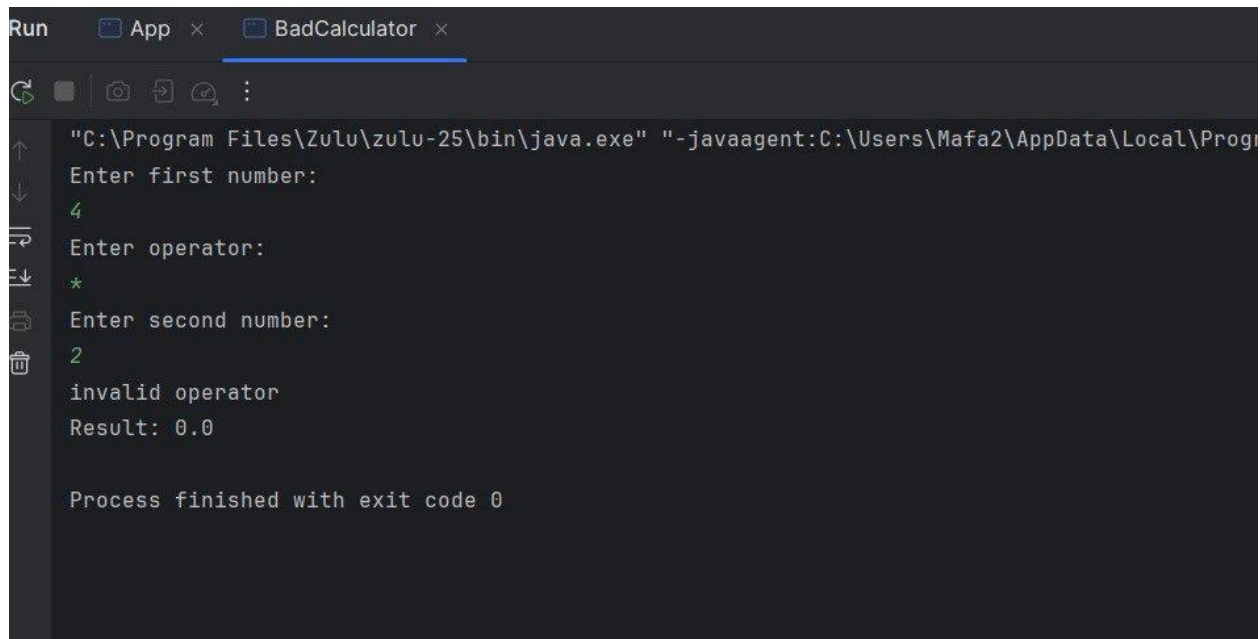
5. Demonstration of Defects

Case 1 – Wrong Result

Input:

```
Run App x BadCalculator x
C:\Program Files\Zulu\zulu-25\bin\java.exe "-javaagent:C:\Users\Mafa2\AppData\Local\Progr
Enter first number:
4
Enter operator:
*
Enter second number:
2
```

Output:

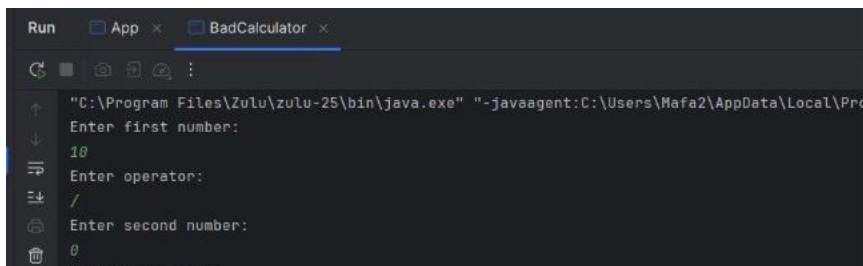


```
Run  App x  BadCalculator x
"C:\Program Files\Zulu\zulu-25\bin\java.exe" "-javaagent:C:\Users\Mafa2\AppData\Local\Progr
Enter first number:
4
Enter operator:
*
Enter second number:
2
invalid operator
Result: 0.0

Process finished with exit code 0
```

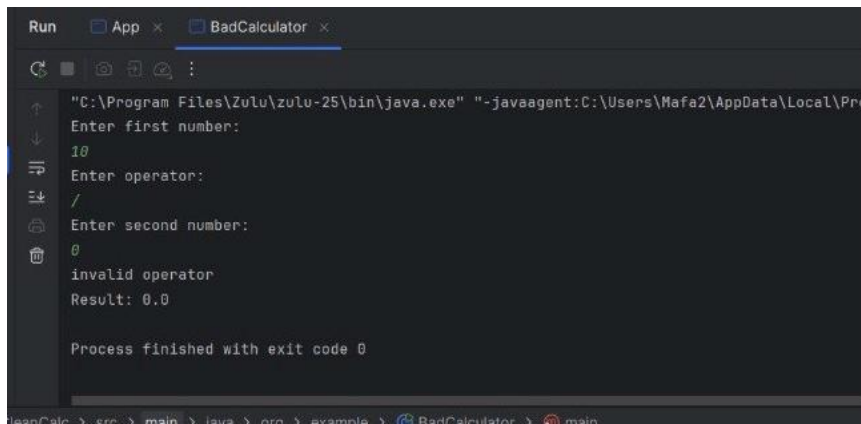
Case 2 – Division by Zero

Input:



```
Run  App x  BadCalculator x
"C:\Program Files\Zulu\zulu-25\bin\java.exe" "-javaagent:C:\Users\Mafa2\AppData\Local\Progr
Enter first number:
10
Enter operator:
/
Enter second number:
0
invalid operator
```

Output:



```
Run  App x BadCalculator x
"C:\Program Files\Zulu\zulu-25\bin\java.exe" "-javaagent:C:\Users\Mafa2\AppData\Local\Pro
Enter first number:
10
Enter operator:
/
Enter second number:
0
invalid operator
Result: 0.0

Process finished with exit code 0
```

The above results clearly indicate incorrect logic and missing exception handling.

6. Refactoring and Fixes

After analyzing the flaws, the code was refactored following SQA principles of modularity, clarity, and reliability.

Improvement	Description
Validation Added	Implemented input checks and <code>ArithmeticException</code> for divide-by-zero cases.
Modular Design	Split UI and logic into separate classes (<code>App.java</code> and <code>Calculator.java</code>).
Clean Naming	Improved naming conventions for readability and maintainability.
Testing Integration	Added JUnit-based tests to automatically verify all operations.
Standard Packaging	Organized all files under <code>org.example</code> package and managed build using Maven.

7. Refactored Code (Clean Version)

Calculator.java

```
1
2  package org.example;
3
4  public class Calculator { no usages
5      public double add(double a, double b) { return a + b; }
6      public double subtract(double a, double b) { return a - b; } no usages
7      public double multiply(double a, double b) { return a * b; } no usages
8      public double divide(double a, double b) { no usages
9          if (b == 0) throw new ArithmeticException("Cannot divide by zero");
10         return a / b;
11     }
12 }
```

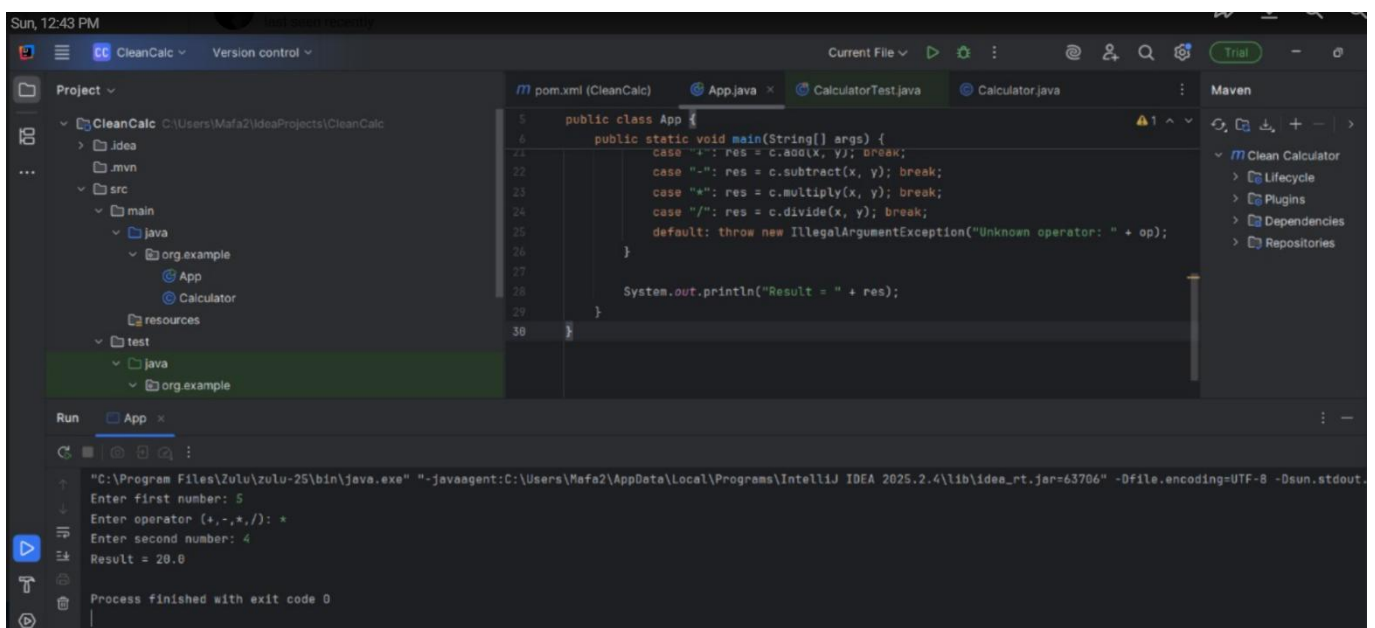
App.java

```
1 package org.example;
2
3 import java.util.Scanner;
4
5 public class App {
6     public static void main(String[] args) {
7         Calculator c = new Calculator();
8         Scanner sc = new Scanner(System.in);
9
10        System.out.print("Enter first number: ");
11        double x = sc.nextDouble();
12
13        System.out.print("Enter operator (+,-,*,/): ");
14        String op = sc.next();
15
16        System.out.print("Enter second number: ");
17        double y = sc.nextDouble();
18
19        double res;
20        switch (op) {
21            case "+": res = c.add(x, y); break;
22            case "-": res = c.subtract(x, y); break;
23            case "*": res = c.multiply(x, y); break;
24            case "/": res = c.divide(x, y); break;
25            default: throw new IllegalArgumentException("Unknown operator: " + op);
26        }
27
28        System.out.println("Result = " + res);
29    }
30 }
```

Figure 3: Refactored and improved code version.

8. Execution Results (After Fixes)

Case 1: Normal Operation



```
Sun, 12:43 PM
CleanCalc
Version control
Current File
pom.xml (CleanCalc) App.java CalculatorTest.java Calculator.java
Maven
Project
CleanCalc
C:\Users\Mafa2\IdeaProjects\CleanCalc
Idea
.mvn
src
main
java
org.example
App
Calculator
resources
test
java
org.example
Run
App
"C:\Program Files\Zulu\zulu-25\bin\java.exe" "-javaagent:C:\Users\Mafa2\AppData\Local\Programs\IntelliJ IDEA 2025.2.4\lib\idea_rt.jar=63706" -Dfile.encoding=UTF-8 -Dsun.stdout.
Enter first number: 5
Enter operator (+,-,*,/): +
Enter second number: 4
Result = 20.0
Process finished with exit code 0
```

Case 2 – Division by Zero

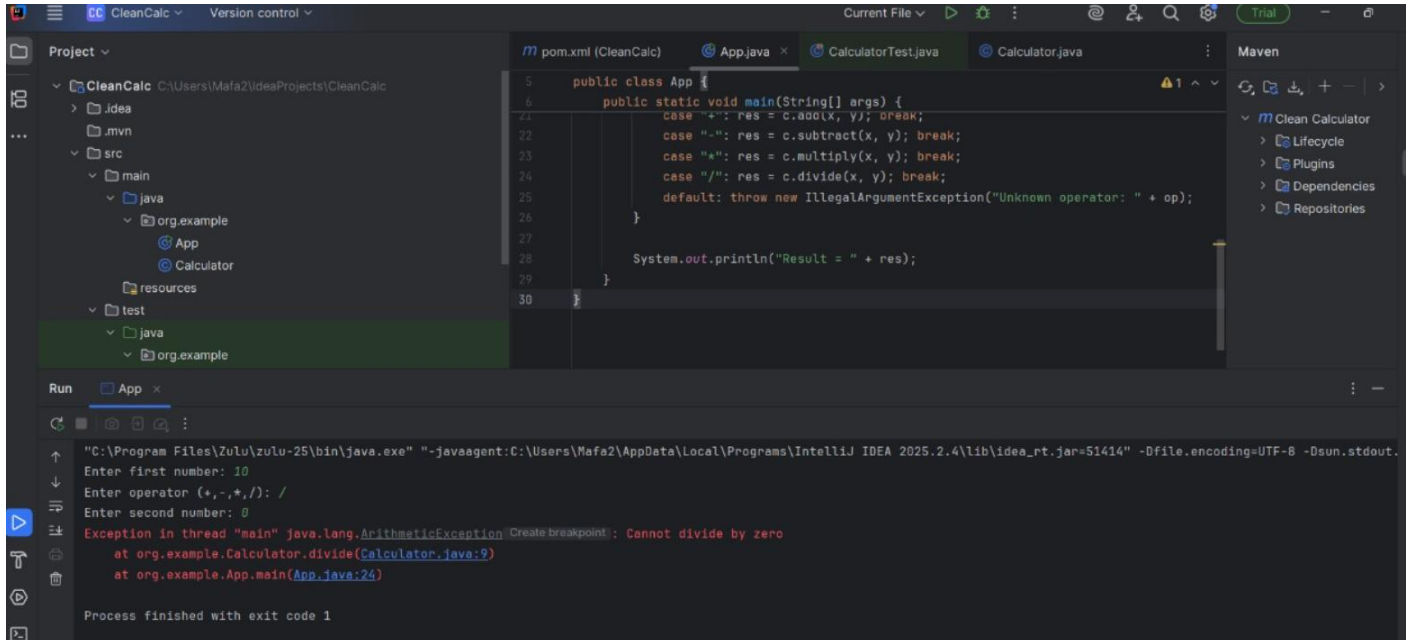


Figure 4: Output after improvements showing valid and safe execution.

9. Testing Results (JUnit Verification)

To ensure correctness, automated unit tests were written for each calculator method.

Test Case	Expected	Result
Addition	$2 + 3 = 5.0$	Passed
Subtraction	$2 - 3 = -1.0$	Passed
Multiplication	$2 * 3 = 6.0$	Passed
Division	$5 / 2 = 2.5$	Passed
Division by Zero	Exception thrown	Passed

All tests passed successfully, as shown in the following figure.

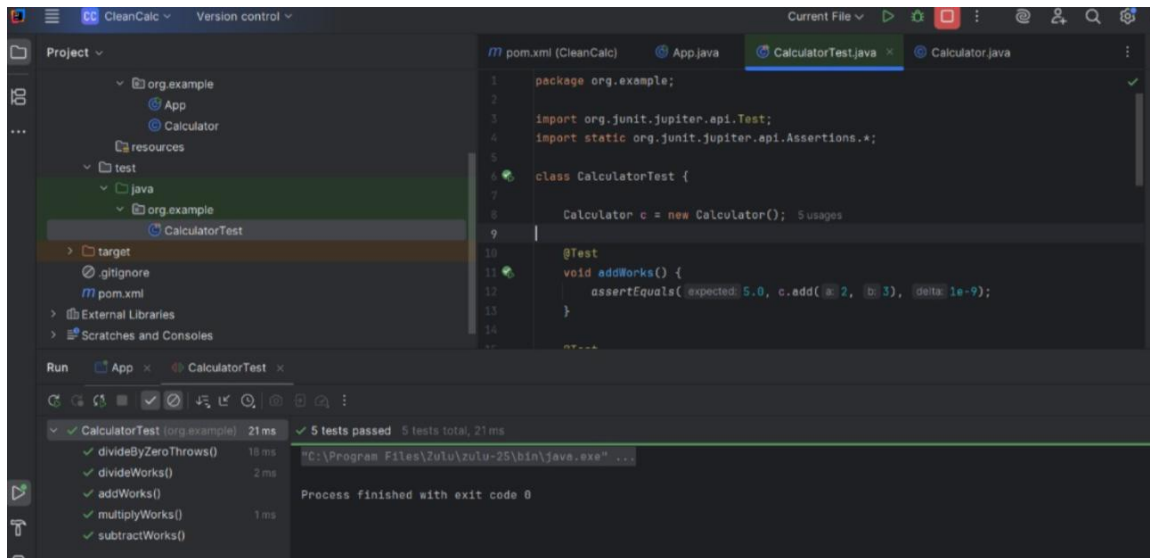


Figure 5: Screenshot of JUnit results (Green bar – all tests passed).

10. Discussion

The implementation of SQA principles significantly improved software quality.

The project now includes verification through unit tests and validation through user input handling.

Reliability increased since the program no longer produces incorrect or undefined results.

Maintainability improved because code is modular and easier to modify or extend.

Reusability of the logic layer (Calculator.java) allows it to be integrated into other applications.

These enhancements directly align with the objectives of SW340, translating theory into practical application.

11. Quality Comparison (Before vs After)

Metric	Before Refactoring	After Refactoring
Code Structure	All logic in one file	Modular, separated logic
Readability	Poor	Clear and consistent
Error Handling	None	Implemented
Testing	None	5 automated tests
Maintainability	Low	High
Reliability	Unreliable results	Verified correctness

Figure 6: Comparison table of software quality metrics.

12. Conclusion

This project successfully demonstrates how applying Software Quality Assurance principles can transform a poorly structured and error-prone program into a clean, maintainable, and reliable software product.

Through systematic analysis, refactoring, and testing, the calculator application now meets modern software quality standards. The lessons learned reinforce the importance of SQA activities such as validation, verification, testing, and continuous improvement in software development.

13. References

IEEE Std 730-2014, Software Quality Assurance Processes

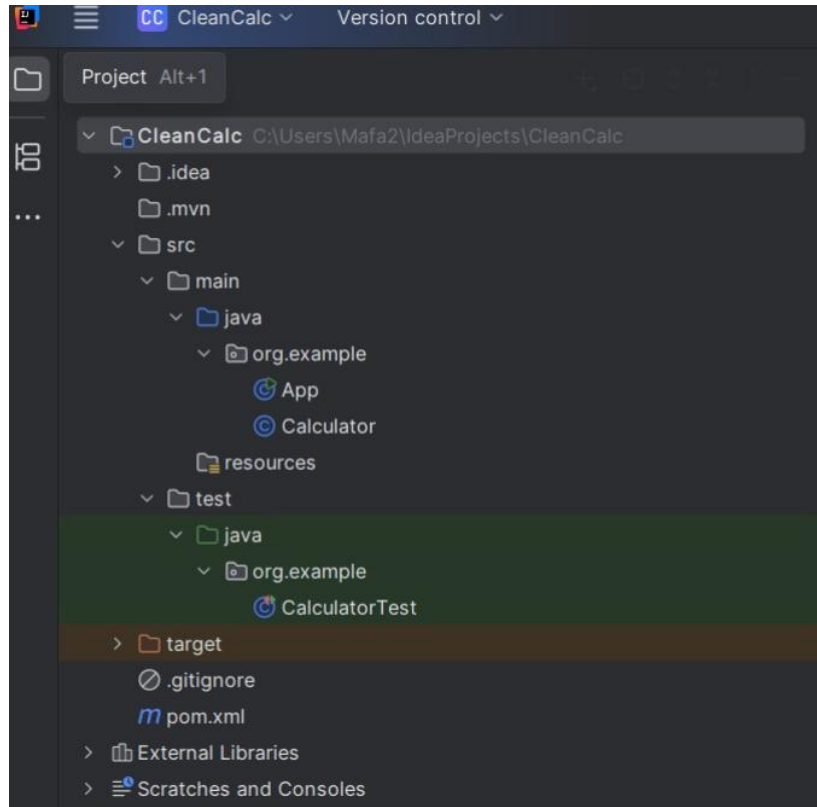
JUnit 5 Documentation, <https://junit.org/junit5/>

Apache Maven Documentation, <https://maven.apache.org/>

Oracle Java SE 17 Developer Guide

14. Appendix – Screenshots

1. Project Structure (CleanCalc)



2. pom.xml Configuration

```
Current File
pom.xml (CleanCalc) x App.java CalculatorTest.java BadCalculator.java Calculator.java
1 <project xmlns="http://maven.apache.org/POM/4.0.0"
2   xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3   xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
4   <modelVersion>4.0.0</modelVersion>
5
6   <groupId>qa</groupId>
7   <artifactId>CleanCalc</artifactId>
8   <version>1.0.0</version>
9   <name>Clean Calculator</name>
10
11   <properties>
12     <maven.compiler.source>17</maven.compiler.source>
13     <maven.compiler.target>17</maven.compiler.target>
14     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
15     <junit.jupiter.version>5.10.2</junit.jupiter.version>
16   </properties>
17
18   <dependencies>
19     <!-- JUnit 5 -->
20     <dependency>
21       <groupId>org.junit.jupiter</groupId>
22       <artifactId>junit-jupiter</artifactId>
23       <version>${junit.jupiter.version}</version>
24       <scope>test</scope>
25     </dependency>
26   </dependencies>
27
28   <build>
29     <plugins>
30       <plugin>
31         <groupId>org.apache.maven.plugins</groupId>
```

```
ec 1 pom.xml (CleanCalc) x App.java CalculatorTest.java BadCalculator.java Calculator.java
17 <project xmlns="http://maven.apache.org/POM/4.0.0"
18
19   <dependencies>
20     <!-- JUnit 5 -->
21     <dependency>
22       <groupId>org.junit.jupiter</groupId>
23       <artifactId>junit-jupiter</artifactId>
24       <version>${junit.jupiter.version}</version>
25       <scope>test</scope>
26     </dependency>
27   </dependencies>
28
29   <build>
30     <plugins>
31       <plugin>
32         <groupId>org.apache.maven.plugins</groupId>
33         <artifactId>maven-surefire-plugin</artifactId>
34         <version>3.2.5</version>
35         <configuration>
36           <useModulePath>false</useModulePath>
37         </configuration>
38       </plugin>
39     </plugins>
40   </build>
41 </project>
```

3. Console Output – Old Version

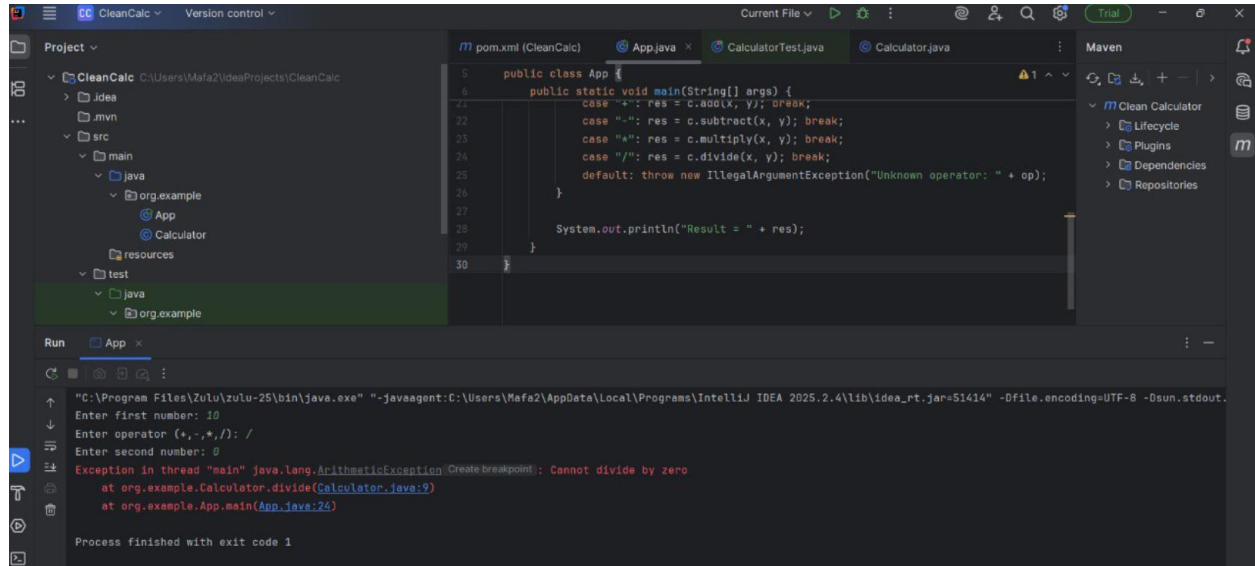

```
4 public class BadCalculator {
6     public static void main(String args[]) {
12
13         System.out.println("Enter first number: ");
14         num1 = s.nextInt();
15
16         System.out.println("Enter operator: ");
17         operator = s.next(); //
18
19         System.out.println("Enter second number: ");
20         num2 = s.nextInt(); //
21
22         if (operator == "+") {
23             result = num1 + num2;
24         }
25     }
26 }
```

Run App x BadCalculator x

"C:\Program Files\Zulu\zulu-25\bin\java.exe" "-javaagent:C:\Users\Mafa2\AppData\Local\Progr
Enter first number:
10
Enter operator:
/
Enter second number:
0
invalid operator
Result: 0.0

Process finished with exit code 0

4. Console Output – New Version

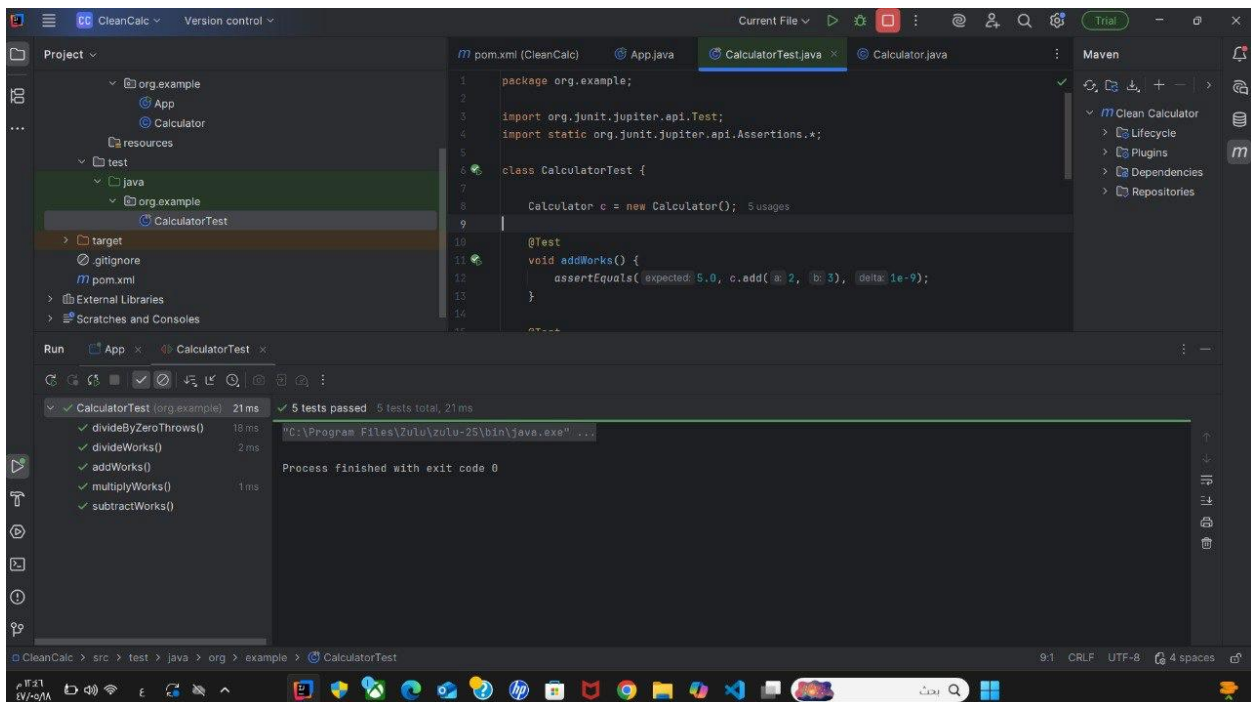


The screenshot shows the IntelliJ IDEA interface with the 'CleanCalc' project. The 'App.java' file is open, displaying a simple calculator application. The console output shows the program running and throwing an `Exception in thread "main" java.lang.ArithmeticException: Cannot divide by zero` at `org.example.Calculator.divide(Calculator.java:9)` and `at org.example.App.main(App.java:24)`. The process finished with exit code 1.

```
public class App {  
    public static void main(String[] args) {  
        case "+": res = c.add(x, y); break;  
        case "-": res = c.subtract(x, y); break;  
        case "*": res = c.multiply(x, y); break;  
        case "/": res = c.divide(x, y); break;  
        default: throw new IllegalArgumentException("Unknown operator: " + op);  
    }  
    System.out.println("Result = " + res);  
}
```

```
Process finished with exit code 1
```

5. JUnit Test Results



The screenshot shows the IntelliJ IDEA interface with the 'CleanCalc' project. The 'CalculatorTest.java' file is open, displaying a JUnit test class. The console output shows the test results: `5 tests passed` and `5 tests total, 21 ms`. The process finished with exit code 0.

```
package org.example;  
  
import org.junit.jupiter.api.Test;  
import static org.junit.jupiter.api.Assertions.*;  
  
class CalculatorTest {  
    Calculator c = new Calculator();  
  
    @Test  
    void addWorks() {  
        assertEquals(5.0, c.add(2, 3), delta(1e-9));  
    }  
}
```

```
Process finished with exit code 0
```

6. SDK Settings (JDK 17)

